

1. Descrição Detalhada do Problema

1.1 Contexto

O controle financeiro pessoal é uma prática essencial para a saúde econômica de qualquer indivíduo. No entanto, muitas pessoas enfrentam dificuldades para manter um registro consistente de suas receitas e despesas. As soluções atualmente disponíveis costumam se dividir em dois extremos: ou são planilhas manuais complexas que exigem muito tempo de manutenção, ou são aplicativos em nuvem repletos de funcionalidades excessivas, que exigem conexão constante com a internet e levantam preocupações sobre a privacidade e o compartilhamento de dados financeiros sensíveis.

1.2 O Problema

A falta de uma ferramenta equilibrada resulta na perda de visibilidade financeira. Sem um acompanhamento simples, os usuários têm dificuldade em responder a perguntas básicas sobre suas próprias finanças, como: "Qual é o meu saldo atual?", "Quanto gastei com alimentação este mês?" ou "Onde posso reduzir meus gastos?". A ausência de um sistema rápido, privado e de baixa fricção desmotiva o usuário a registrar suas movimentações rotineiras, levando ao descontrole financeiro.

1.3 A Justificativa e a Solução

O **Sistema de Controle de Gastos Pessoais** surge para preencher essa lacuna, oferecendo uma solução com foco total em simplicidade e uso local. Ao optar por uma arquitetura onde a aplicação roda localmente no computador do usuário (acessível pelo navegador) e os dados são armazenados em um banco de dados local SQLite, o projeto garante:

- **Privacidade Absoluta:** As informações financeiras nunca saem da máquina do usuário.
- **Disponibilidade:** O sistema pode ser utilizado independentemente de conexão com a internet após ser iniciado localmente.
- **Simplicidade de Uso:** Uma interface web direta e intuitiva, focada apenas nas funcionalidades essenciais de cadastro, categorização e filtragem.

Além de resolver o problema do usuário final, o desenvolvimento deste sistema atua como um projeto acadêmico estruturado. Ele permite a aplicação prática de conceitos modernos de engenharia de software, englobando a construção de uma API REST (utilizando Python e FastAPI), modelagem de dados, testes automatizados, containerização com Docker e documentação técnica padronizada.

2. Levantamento de Requisitos

2.1 Requisitos Funcionais (RF)

Os requisitos funcionais descrevem as ações e comportamentos que o sistema deve fornecer ao usuário.

- **RF01 - Cadastro de Transações:** O sistema deve permitir o cadastro de novas movimentações financeiras, sendo elas receitas ou despesas. Cada transação deve conter obrigatoriamente: valor, data, tipo (receita/despesa) e a categoria associada.
- **RF02 - Gerenciamento de Categorias Personalizadas:** O sistema deve permitir a categorização dos gastos. O usuário deve ter total liberdade para realizar o gerenciamento completo (Criar, Ler, Atualizar e Excluir) de suas próprias categorias (ex: "contas fixas", "investimentos", "alimentação", "gasolina"), não ficando restrito a um modelo estático.
- **RF03 - Consulta de Transações:** O sistema deve permitir listar todas as movimentações financeiras cadastradas. Também deve permitir a consulta detalhada de uma movimentação específica através de seu identificador único (ID).
- **RF04 - Exclusão de Transações:** O sistema deve permitir que o usuário exclua uma movimentação financeira específica, utilizando o seu ID.
- **RF05 - Filtros de Busca:** O sistema deve disponibilizar a funcionalidade de filtrar as movimentações financeiras cadastradas buscando por categoria e por período específico (Mês e Ano, para evitar conflito de dados históricos).
- **RF06 - Consulta de Saldo:** O sistema deve calcular e permitir a consulta do saldo financeiro atual do usuário (Receitas - Despesas).
- **RF07 - Resumo Financeiro:** O sistema deve gerar e exibir um resumo financeiro consolidado, apresentando claramente o total acumulado de receitas, o total de despesas e o saldo final.
- **RF08 - Interface de Usuário:** O sistema deve disponibilizar uma interface web simples e intuitiva para que o usuário possa acessar todas as funcionalidades citadas diretamente pelo navegador.
- **RF09 - Exportação de Relatório:** O sistema deve permitir a exportação das transações (seja a lista completa ou o resultado de um filtro) para um arquivo CSV, possibilitando o uso dos dados em planilhas externas.

2.2 Requisitos Não Funcionais (RNF)

Os requisitos não funcionais especificam os critérios técnicos, de desempenho, ambiente e restrições de arquitetura do sistema.

- **RNF01 - Linguagem e Backend:** A API REST e o servidor web da aplicação devem ser desenvolvidos utilizando a linguagem Python, em conjunto com o framework FastAPI.
- **RNF02 - Persistência de Dados e ORM:** O armazenamento dos dados deve ser feito localmente utilizando o banco de dados SQLite, de modo que as informações continuem salvas mesmo após a aplicação ser fechada ou o computador desligado. A modelagem e o acesso ao banco devem ser realizados utilizando a biblioteca SQLAlchemy. A persistência do banco SQLite deve ser garantida através do mapeamento de Volumes do Docker, evitando perda de dados caso o container seja recriado.
- **RNF03 - Ambiente de Execução e Hospedagem Local:** O sistema deve operar de forma inteiramente local, acessível no próprio computador do usuário através do endereço <http://localhost:8000>.
- **RNF04 - Containerização e Padronização:** O ambiente de desenvolvimento e execução deve ser padronizado através do uso de Docker.

- **RNF05 - Garantia de Qualidade e Testes:** A aplicação deve possuir uma suíte de testes automatizados construída com a ferramenta Pytest. A estrutura de testes (Test Harness) deve cobrir os cenários principais e também os casos de borda (edge cases).
- **RNF06 - Controle de Versão e Organização:** O versionamento do código-fonte será realizado via GitHub. O fluxo de trabalho deve exigir o uso de Pull Requests e revisões de código antes das integrações (merges) com as branches principais.
- **RNF07 - Frontend:** A interface web deve ser construída utilizando tecnologias fundamentais: HTML, CSS e JavaScript, sem a necessidade de frameworks complexos, garantindo leveza e aderência à proposta.
- **RNF08 - Documentação da API:** O sistema deve disponibilizar a documentação interativa da API REST (Swagger/OpenAPI) gerada automaticamente pelo FastAPI, acessível via navegador (ex: [/docs](#)), facilitando a consulta dos endpoints e os testes da equipe.
- **RNF09 - Padronização de Erros:** A API deve retornar códigos de status HTTP semânticos (ex: 200 para sucesso, 201 para criado, 400 para dados inválidos, 404 para não encontrado) e mensagens de erro estruturadas em formato JSON padronizado.

3. Regras de Negócio (RN)

3.1 Regras de Transações

- **RN01 - Campos Obrigatórios:** Nenhuma transação pode ser salva sem que os campos "Valor", "Data", "Tipo" (Receita ou Despesa) e "Categoria" sejam informados. Se faltar algum, a API deve recusar a requisição (Erro 400).
- **RN02 - Validação de Valores Absolutos:** O valor de qualquer movimentação (seja receita ou despesa) deve ser estritamente maior que zero (> 0.00). O sistema não deve aceitar valores negativos ou zerados no momento do cadastro (o fato de subtrair do saldo será definido exclusivamente pelo campo "Tipo = Despesa").
- **RN03 - Formatação e Validação de Datas:** A data da transação deve obrigatoriamente seguir o padrão ISO 8601 ($YYYY-MM-DD$). O sistema deve rejeitar datas inexistentes (como 30 de fevereiro).

3.2 Regras de Categorias (Categorias Personalizadas)

- **RN04 - Unicidade de Categorias:** O sistema não deve permitir o cadastro de duas categorias com o mesmo nome exato (ignorando letras maiúsculas e minúsculas). Isso evita confusões no momento de gerar os resumos e filtros.
- **RN05 - Integridade na Exclusão de Categorias:** Se o usuário tentar excluir uma categoria que já está vinculada a uma ou mais transações existentes, o sistema deve **bloquear a exclusão** e retornar um erro (ex: "Não é possível excluir uma categoria em uso"). O usuário deve primeiro excluir ou alterar as transações associadas a ela. *(Nota: Essa é uma regra clássica de integridade referencial de banco de dados).*
- **RN06 - Categoria Padrão/Default:** Ao inicializar o banco de dados pela primeira vez, o sistema pode criar algumas categorias padrão (ex: "Alimentação", "Salário",

"Outros") para que o usuário não comece com o sistema totalmente vazio, podendo editá-las ou apagá-las posteriormente (respeitando a RN05).

3.3 Regras de Saldo e Resumo Financeiro

- **RN07 - Permissão de Saldo Negativo:** O sistema deve calcular o saldo total somando as Receitas e subtraindo as Despesas. É permitido que o saldo final fique negativo (representando dívidas ou cheque especial), não devendo haver bloqueio de novos cadastros de despesas caso o saldo esteja zerado ou menor que zero.
- **RN08 - Comportamento de Filtros Vazios:** Caso o usuário aplique um filtro de mês/ano ou categoria que não possua nenhuma transação correspondente, o sistema não deve retornar erro. Deve retornar uma lista vazia (HTTP 200 com array `[]`) e os totais de resumo financeiro zerados.

4. Especificação de Entradas e Saídas (Contratos da API)

4.1 Domínio: Categorias

A. Cadastrar Categoria

- **Rota:** POST /categorias
- **Descrição:** Cria uma nova categoria personalizada.
- **Request (Body):** { "nome": "Contas Fixas" }
- **Response (201 Created):** { "id": 1, "nome": "Contas Fixas" }

B. Listar Categorias

- **Rota:** GET /categorias
- **Descrição:** Retorna todas as categorias cadastradas.
- **Response (200 OK):** [{ "id": 1, "nome": "Contas Fixas" }, { "id": 2, "nome": "Alimentação" }]

C. Excluir Categoria

- **Rota:** DELETE /categorias/{id}
- **Descrição:** Remove uma categoria (respeitando a regra de bloqueio caso esteja em uso).
- **Response (204 No Content):** Sem corpo de resposta.
- **Erro (400 Bad Request):** { "erro": "Categoria em uso." }

D. Atualizar Categoria

- **Rota:** PUT /categorias/{id}
- **Descrição:** Atualiza o nome de uma categoria existente.
- **Request (Body):** { "nome": "Despesas Fixas" }
- **Response (200 OK):** { "id": 1, "nome": "Despesas Fixas" }
- **Erro (404 Not Found):** { "erro": "Categoria não encontrada." }

4.2 Domínio: Transações

A. Cadastrar Transação

- **Rota:** POST /transacoes
- **Descrição:** Cadastrar receita ou despesa.
- **Request (Body):**

```
{
  "valor": 150.50,
  "data": "2026-09-10",
  "tipo": "despesa",
  "categoria_id": 2,
  "descricao": "Compra no mercado"
}
```
- **Response (201 Created):**

```
{
  "id": 101,
  "valor": 150.50,
  "data": "2026-09-10",
  "tipo": "despesa",
  "categoria_id": 2,
  "descricao": "Compra no mercado"
}
```

B. Listar e Filtrar Transações

- **Rota:** GET /transacoes
- **Query Parameters (Opcionais):**
?mes=09&ano=2026&categoria=2&limit=50&offset=0
- **Descrição:** Listar movimentações e aplicar filtros.
- **Response (200 OK):**

```
[[
  {
    "id": 101,
    "valor": 150.50,
    "data": "2026-09-10",
    "tipo": "despesa",
    "categoria": { "id": 2, "nome": "Alimentação" },
    "descricao": "Compra no mercado"
  }
]]
```

C. Consultar Transação Específica

- **Rota:** GET /transacoes/{id}
- **Descrição:** Consultar uma movimentação.
- **Response (200 OK):** Mesmo formato de objeto único listado acima.
- **Erro (404 Not Found):** { "erro": "Transação não encontrada." }

D. Atualizar Transação

- **Rota:** PUT /transacoes/{id}
- **Descrição:** Atualiza os dados de uma movimentação financeira existente.
- **Request (Body):**
 - {
 - "valor": 160.00, "data": "2026-09-11", "tipo": "despesa", "categoria_id": 2, "descricao": "Compra no mercado (valor corrigido)"
 - }
- **Response (200 OK):**
 - {
 - "id": 101, "valor": 160.00, "data": "2026-09-11", "tipo": "despesa", "categoria_id": 2, "descricao": "Compra no mercado (valor corrigido)"
 - }
- **Erro (404 Not Found):** { "erro": "Transação não encontrada." }

E. Excluir Transação

- **Rota:** DELETE /transacoes/{id}
- **Descrição:** Excluir uma movimentação.
- **Response (204 No Content):** Sem corpo de resposta.

4.3 Domínio: Resumos e Exportação

A. Consultar Saldo

- **Rota:** GET /saldo
- **Descrição:** Consultar saldo atual.
- **Response (200 OK):** { "saldo_atual": 1250.00 }

B. Consultar Resumo Financeiro

- **Rota:** GET /resumo
- **Query Parameters (Opcionais):** ?mes=09&ano=2026
- **Descrição:** Consultar resumo financeiro do período.
- **Response (200 OK):**
 - {
 - "total_receitas": 3000.00,
 - "total_despesas": 1750.00,
 - "saldo_periodo": 1250.00
 - }

C. Exportar Relatório

- **Rota:** GET /transacoes/exportar
- **Query Parameters (Opcionais):** ?mes=09&ano=2026
- **Descrição:** Retorna um arquivo .csv com as movimentações filtradas.
- **Response (200 OK):** Download do arquivo "relatorio_gastos.csv".

5. Refinamento por Feedback e Alterações

- **Ajuste 1:** Adição de endpoints de Atualização (PUT) para Categorias e Transações visando permitir a edição de registros após revisão inicial do grupo.
- **Ajuste 2:** Inclusão de paginação (limit/offset) no endpoint de listagem de transações para evitar problemas de performance no frontend.